

数组的操作

给定一个长度为 n 的数组，再给定 m 组操作。操作只有两种，在末尾删除一个元素，在末尾插入一个元素。操作包含两个数字，第一个数字，1表示增加，0表示删除。第二个数字表示增加的数(如果是删除则没有)。如果中途没有数可以删除。则输出NO。否则输出最终的数组（如果最终没有元素，输出0）。

输出：

1 2 3 4 3

输入：

4

1 2 3 4

3

1 3

1 2

0

$n \leq 10000, m \leq 10000$

数组的操作

因为每次都是在数组的末尾做增加删除操作。所以我们要必须要知道数组末尾的位置。如果是插入一个数。数组末尾的位置+1. 如果是删除一个数。数组末尾的位置-1.

同时，在做删除操作的时候。我们还需要判断此时数组是否为空。即数组末尾位置是否为0. 如果为0，表示不可以再删除了

在操作结束的时候，如果数组为空，则输出0,。

对于这种在数组的末尾做删除，增加操作的题，我们可以用一种数据结构来表示：

栈

生活中的栈



我们依次放入多个羽毛球，先放入的羽毛球在下面，后放入的羽毛球在上面。每次使用羽毛球的时候都是先拿出最上面的。每次放入的羽毛球都是在最上面



手枪弹夹装入子弹，先装入的子弹在下面，后装入的子弹在上面。射击时先射出最上面的子弹。装弹时新装入的子弹永远在最上面。

栈 (STACK)

栈 (Stack)：一种后入先出 (LIFO) 的数据结构。

操作要求：只允许在末端（一端）进行删除和插入操作

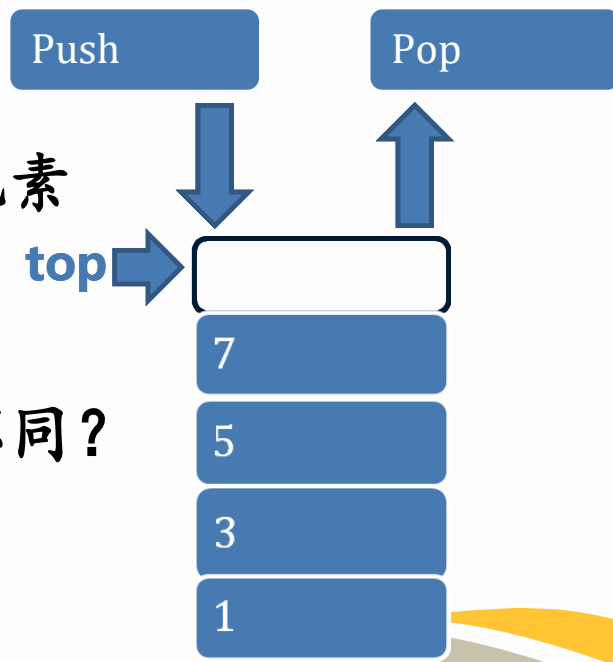
基本操作：

入栈 (Push)：往栈顶添加新的元素

出栈 (Pop)：删除栈顶元素。

相比较于普通的数组，栈有什么不同？

栈顶指针



数组模拟栈

栈的定义：

先定义一个数组：`int stack[100];`

再定义一个栈顶指针，即栈顶元素的位置：`int top=0;`
`top`表示栈顶元素的位置。`top==0`表示栈为空

入栈 (Push)：往栈顶添加元素 X

`top++;`

`stack[top]=X;`

出栈 (Pop)：弹出栈顶元素并且赋值给变量 X

`X=stack[top];`

`top--;`

判断栈是否为空：

`if (top==0) ...;`

栈的有限长度就是从
起点到栈顶的位置

括号匹配

输入一串带括号的表达式，判断输入的表达式是否合理。
即判断括号是否匹配。为了简化题目，只输出左右括号。
如果表达式合理，输出Y E S，如果不合理，输出N O。

输入：

()()

输出：

Y E S

输入：

()) (

输出：

N O

输入字符串长度 ≤ 1000 ;

注意下标起点

括号匹配

做法一：直接比较左括号和右括号的数量。

反例：）（

解题思路：当出现一个右括号的时候，就用当前最后一个左括号来匹配。如果出现右括号的时候前面没有未匹配的左括号。那么表示输出表达式不合理。如果匹配结束还有多余的左括号。那么也表示表达式不合理。

当出现一个左括号的时候：

左括号入栈： $top++$ ；

当出现一个右括号的时候：

左括号出栈与之匹配： $top--$ ；

如果没有左括号与之匹配或者左括号过剩：

输入表达式不合理

括号匹配

```
top=-1;
for(int i=0;i<l;i++)
{
    if(s[i]=='(')                top++; //左括号入栈
    else                          //stack[top]=s[i];
    {
        if(top==-1) //表示栈已经为空，没有左括号
        {
            printf("NO");
            return 0;
        }
        top--; //如果栈不为空，左括号出栈与之匹配
    }
}
if(top==-1) //左括号和右括号数量一定要匹配
printf("YES");
else //如果有多余的左括号
printf("NO");
```

入栈的只有左括号。可以不需要记录

top++; //左括号入栈

//stack[top]=s[i];

if(top==-1) //表示栈已经为空，没有左括号

{
 printf("NO");
 return 0;

}
top--; //如果栈不为空，左括号出栈与之匹配

}
if(top==-1) //左括号和右括号数量一定要匹配
printf("YES");
else //如果有多余的左括号
printf("NO");

STL 栈 (C++)

在C++的STL函数库中，自带一些基本的数据结构。其中就包括我们今天学习的栈，我们可以直接调用

头文件：`#include<stack>`

定义：`stack<int> s;`

取栈顶元素：`s.top();`

入栈(压栈)：`s.push(m);`

出栈(弹栈)：`s.pop();`

判断栈是否为空：`s.empty();`

栈中元素有效个数：`s.size();`

定义一个数据类型为int的栈 s，其中存储空间自动增长，不需要设置

入栈和出栈操作时，不需要更改栈顶元素的位置，因为他会自动更新

如果栈为空，返回1，如果栈不为空，返回0

STL 栈(C++)

```
#include<stack>
stack<char> s;//STL库栈, 自动增长型
int main()
{
    输入字符串
    for(int i=0;i<1;i++)
    {
        if(s1[i]=='(')    s.push(s1[i]); //入栈
        else
        {
            if(s.empty())//如果栈为空成立
            {
                printf("NO");    return 0;
            }
            s.pop();//出栈, 栈顶指针自动减一
        }
    }
    if(s.empty())//如果最终栈为空
    else    printf("NO");
    printf("YES");
}
```

表达式求值

输入一行表达式，只包含+,-,*,/,求表达式的结果。（暂时不含括号，并且输入的都是个位数）。

输入：

$1+2*3/4-5$

输出：

-3

提示：由于+，-和*，/的优先级不一样，是先算乘除，再算加减，所以需要分开考虑。

括号匹配

输入的表达式中含有三种括号：(), [], {}, 判断表达式是否合理，注意，括号由里到外的顺序只能是小括号，中括号，大括号。但是相同括号可以重叠，比如 [[]] 是可以的。

输入：

[{[]}]

输出：

NO

输入：

{[]}

输出：

YES